

漏洞分析报告

漏洞名称	CSRF Token 验证取决于请求方法
测试平台	PortSwigger Web Security Academy
漏洞类型	CSRF（跨站请求伪造）
风险等级	中危
测试时间	2026 年

一、漏洞原理

CSRF（Cross-Site Request Forgery，跨站请求伪造）是一种利用用户已认证的会话状态，诱使其浏览器向目标网站发送非预期请求的攻击方式。攻击者构造恶意页面，当已登录的用户访问该页面时，浏览器会自动携带目标站点的 **Cookie** 发送请求，从而在用户不知情的情况下执行敏感操作。

为防御 **CSRF** 攻击，应用程序通常会在表单中嵌入 **CSRF Token**，并在服务端校验该 **Token** 的有效性。然而，部分应用在实现时存在缺陷：仅对 **POST** 请求校验 **CSRF Token**，而对 **GET** 请求不做校验。攻击者可以将原本的 **POST** 请求转换为 **GET** 请求，将参数拼接到 **URL** 查询字符串中，从而完全绕过 **Token** 校验机制。

二、漏洞危害

攻击者可利用此漏洞在用户不知情的情况下修改其账户绑定邮箱。一旦邮箱被篡改，攻击者可进一步通过密码重置功能接管用户账户，获取账户内的敏感数据、执行资金操作或冒充用户身份进行其他恶意行为。由于攻击仅需诱导用户点击一个链接或访问一个页面即可触发，利用门槛极低，影响范围广泛。

三、复现过程

3.1 测试环境

测试平台：PortSwigger Web Security Academy 靶场

使用工具：Burp Suite、浏览器

3.2 测试步骤

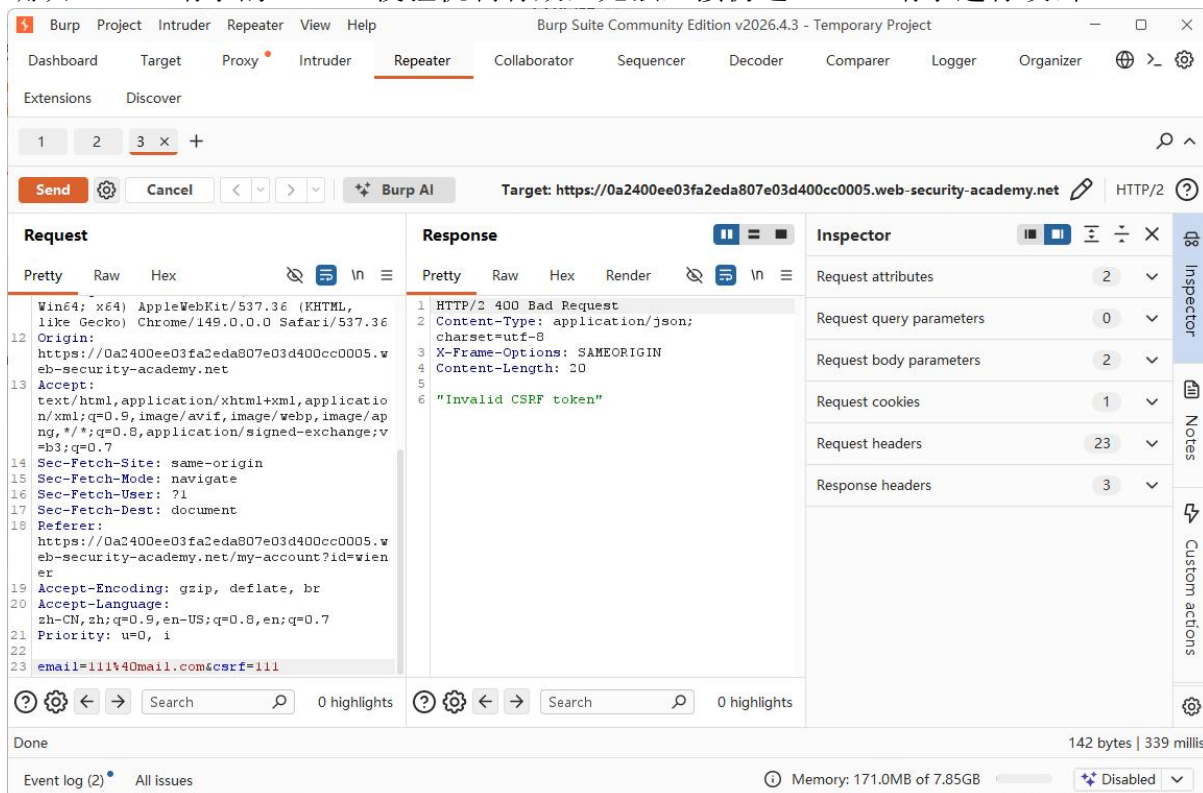
Step 1：确认目标功能和正常请求

登录账户，找到敏感操作，输入 111@mail.com，用 Burp Suite 抓包查看请求确认是 **POST** 请求，参数带 email&csrf

```
3 email=111%40mail.com&csrf=
xP5j3yHCWPscY3qluSXZphZ6DhgjGYUI
```

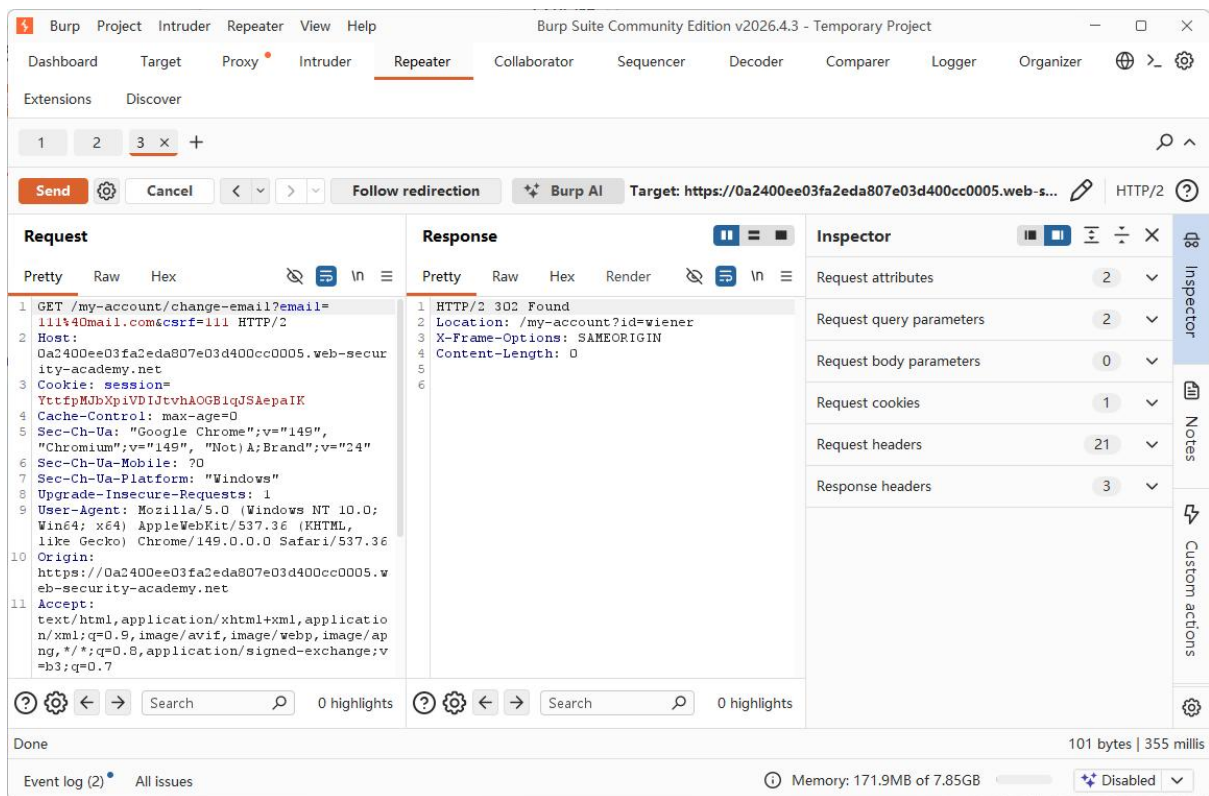
Step 2: 测试 CSRF token 校验

将请求发送至 Burp Suite Repeater，修改 csrf 参数值为无效字符串（如"111"）后重新发送 POST 请求。服务器返回 400 Bad Request，响应内容为"Invalid CSRF token"，确认 POST 请求的 Token 校验机制有效，无法直接伪造 POST 请求进行攻击



Step 3: 测试不同请求方法

既然 POST 请求的 Token 校验有效，尝试测试服务器是否对其他 HTTP 方法也做了同样的校验。在 Burp Suite Repeater 中右键选择 Change request method，将 POST 请求转换为 GET 请求，参数自动从请求体移至 URL 查询字符串，同时保留错误的 csrf 值



服务器返回 302 Found，重定向至/my-account?id=wiener，说明 GET 请求携带无效 csrf 值仍被接受，邮箱修改成功。由此确认：服务器仅对 POST 请求校验 CSRF Token，GET 请求完全未做校验，漏洞存在

Step 4: 构造 exploit 验证利用

确认漏洞可利用后，在靶场提供的 Exploit Server 中构造恶意 HTML 页面。利用 img 标签自动发起 GET 请求的特性，构造如下 Payload：。点击 View exploit 自测，验证 exploit 是否生效

HTTPS
☒

文件：
/exploit

头：
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8

身体：
<html>
<body>

</body>
</html>

访问 **exploit** 页面后返回账户页面，发现邮箱已被修改为 **attacker@evil.com**，**exploit** 生效

我的账户

您的用户名是：wiener

您的邮箱地址是：attacker@evil.com

电子邮件

更新电子邮件

点击 **Deliver exploit to victim** 将恶意页面投递给模拟受害者，受害者访问页面后邮箱同样被篡改，攻击成功，实验完成

四、注意事项

- 1. 本次测试的核心思路是：当一种防御机制生效时，不要立即放弃，而是尝试从其他维度（如请求方法、参数位置、编码方式等）寻找绕过点。
- 2. 在实际渗透测试中，**CSRF** 漏洞的利用需要结合社会工程学手段诱导受害者点击恶意链接，因此 **exploit** 的构造应尽量简洁隐蔽。
- 3. 该漏洞之所以存在，根本原因是开发者在实现 **CSRF** 防御时只考虑了 **POST** 请求，遗漏了 **GET** 等其他 **HTTP** 方法。这提示我们安全防御需要覆盖所有可能的请求路径。

五、修复建议

修复方案	说明
统一校验所有 HTTP 方法	对所有涉及状态变更的请求（无论 GET 、 POST 还是其他方法）统一进行 CSRF Token 校验，避免因遗漏某种请求方法而留下绕过点
限制敏感操作仅接受 POST	在服务端对修改邮箱、密码、权限等敏感接口严格限制只接受 POST 方法，拒绝 GET 请求，从根本上杜绝通过切换方法绕过的可能
配置 SameSite Cookie 属性	将会话 Cookie 的 SameSite 属性设置为 Strict 或 Lax ，阻止跨站请求自动携带 Cookie ，作为 CSRF Token 之外的纵深防御措施

六、参考资料

- PortSwigger Web Security Academy - CSRF: <https://portswigger.net/web-security/csrf>
- OWASP CSRF Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- OWASP Top 10: <https://owasp.org/Top10/>